OULUN AMMATTIKORKEAKOULU

RAG-POHJAISEN CHATBOTIN TOTEUTUS JA ARVIOINTI

Tekoälyratkaisu organisaatiokohtaisen tietopohjan hyödyntämiseen

Lauri Virtanen
Opinnäytetyö AMK
Kevät 2026
Tietojenkäsittelyn tutkinto-ohjelma
Oulun ammattikorkeakoulu

TIIVISTELMÄ

Oulun ammattikorkeakoulu
Tietojenkäsittelyn tutkinto-ohjelma
informaatioteknologia

Tekijä: Lauri Virtanen
Opinnäytetyön otsikko: RAG-pohjaisen chatbotin toteutus ja arviointi
Työn valmistumislukukausi ja -vuosi: kevät 2026
Sivumäärä: 40

Tässä opinnäytetyössä kehitettiin Retrieval-Augmented Generation (RAG) -pohjainen arkkitehtuuri verkkosivulle integroitua tekoälypohjaista chatbottia varten. Työn tavoitteena oli toteuttaa toimiva esimerkkiratkaisu ja arvioida, miten RAG-menetelmä parantaa vastausten laatua, luotettavuutta ja ajantasaisuutta verrattuna perinteisiin kielimallipohjaisiin chatbot-ratkaisuihin.

Työssä toteutettiin järjestelmä, jossa käyttäjän kysymykset muunnettiin vektorimuotoon ja niihin haettiin semanttisesti merkityksellistä kontekstia organisaation omasta tietopankista ennen vastauksen muodostamista kielimallilla. Ratkaisussa hyödynnettiin relaatiotietokantaa vektorihakua varten sekä erillistä offline-prosessia tietopankin sisällön esikäsittelyyn. Chatbot liitettiin verkkosivupohjaiseen käyttöliittymään.

Järjestelmän toimivuutta ja suorituskykyä arvioitiin testikyselyillä tuotantoympäristössä. Tulokset osoittivat, että chatbot tuotti palveluun liittyviin kysymyksiin johdonmukaisia vastauksia ja kykeni rajaamaan aiheeseen kuulumattomat kysymykset. Johtopäätöksenä todetaan, että RAG-arkkitehtuuri tarjoaa toimivan ja kustannustehokkaan tavan yhdistää kielimallit organisaatiokohtaiseen tietoon, ja järjestelmää voidaan jatkossa kehittää esimerkiksi tietopankin hallintaa ja käyttäjäpalautteen hyödyntämistä parantamalla.

ABSTRACT

Oulu University of Applied Sciences
Degree Program in Business Information
Data Processing

Author: Lauri Virtanen
Title of thesis: Design and implementation of RAG based chatbot
Term and year when the thesis was submitted: spring 2026
Number of pages: 40

This thesis focused on the development of a chatbot based on a Retrieval-Augmented Generation (RAG) architecture and its integration into a website. The objective was to implement a functional example solution and to evaluate how the RAG approach improves the quality and reliability of chatbot responses compared to traditional decision tree–based solutions.

The implemented system converted user queries into vector representations and retrieved semantically relevant context from an organization specific knowledge base prior to response generation. The solution utilized a relational database for vector search, an offline preprocessing pipeline for knowledge base preparation, and a predefined prompt structure to guide the language model. The chatbot was integrated into a web-based user interface.

The system was evaluated using test queries in a production environment. The results showed that the chatbot produced consistent, domain-specific answers and was able to restrict responses to the intended context. Performance analysis indicated that database retrieval remained efficient, while response generation accounted for most of the total latency. The results suggest that RAG provides a practical and cost-effective approach for combining large language models with organization specific information.

SISÄLLYS

TIIVISTELMÄ 2
ABSTRACT 3
SISÄLLYS 4
1 JOHDANTO 6
1.1 Tavoitteet ja rajaukset 7
2 TIETOPERUSTA JA KÄSITTEET 9
2.1 Chatbotit 9
2.2 Generatiivinen tekoäly ja suuret kielimallit 11
2.2.1 Generatiivinen tekoäly 11
2.2.2 Suuret kielimallit 11
2.2.3 Tokenit, konteksti ja kustannus 12
2.2.4 Arkkitehtuuri ja vastausten muodostus 12
2.2.5 Hallusinaatiot 13
3 RETRIEVAL-AUGMENTED GENERATION 15
3.1 Data ingestion ja esikäsittely 16
3.2 Chunking 16
3.3 Datan vektorointi ja embeddings 16
3.4 Vektorihaku 17
3.5 LLM ja vastausten muodostus 18
4 TOTEUTUS 19
4.1 Toteutuksen teknologiat ja arkkitehtuuri 19
4.2 Tietokantakerros 20
4.2.1 Pgvector-laajennus ja käyttöönotto 20
4.2.2 Tietopankin rakenne 21
4.2.3 Semanttinen haku tietokantafunktion avulla 22
4.2.4 Hakusuorituskyvyn optimointi vektori-indeksillä 22
4.2.5 Tietoturva ja käyttäjäoikeuksien hallinta 23
4.3 Kysely-vastausprosessi 24
4.3.1 Relevantin kontekstin haku tietokannasta 24
4.3.2 Promptin muodostaminen tietopankkikontekstilla 25

4.3.3 Vastauksen generointi ja keskusteluhistorian hyödyntäminen 26
4.3.4 Prosessin ohjausparametrit 27
4.4 Tietopankin hallinta 27
4.5 Käyttöliittymä 29
5 POHDINTA JA ARVIOINTI 31
5.1 Testausasetelma ja mittarit 31
5.2 Vastauksen laatu ja rajaus 31
5.3 Suorituskyky 32
5.4 Kustannukset ja skaalautuvuus 33
5.5 Kehitysideat 34
6 JOHTOPÄÄTÖKSET 36
LÄHTEET 37

1 JOHDANTO

Tässä opinnäytetyössä kehitettiin RAG-pohjainen arkkitehtuuri, jota hyödynnettiin verkkosivulle integroidussa tekoälypohjaisessa chatbotissa. Tavoitteena oli toteuttaa toimiva esimerkkitoteutus ja arvioida, miten RAG parantaa vastausten laatua sekä mahdollistaa ajankohtaisen ja yrityskohtaisen tiedon yhdistämisen tekoälyyn, joka tuottaa vastauksia luonnollisella kielellä.

Tässä työssä chatbotin vastaukset tuotettiin kielimallilla, eli tekoälymallilla, joka on koulutettu laajoilla tekstiaineistoilla ymmärtämään ja tuottamaan luonnollista kieltä. Tällaiset mallit muodostavat vastauksia aiemman tekstikontekstin perusteella, mutta eivät itsessään sisällä organisaatiokohtaista tai ajantasaista tietoa.

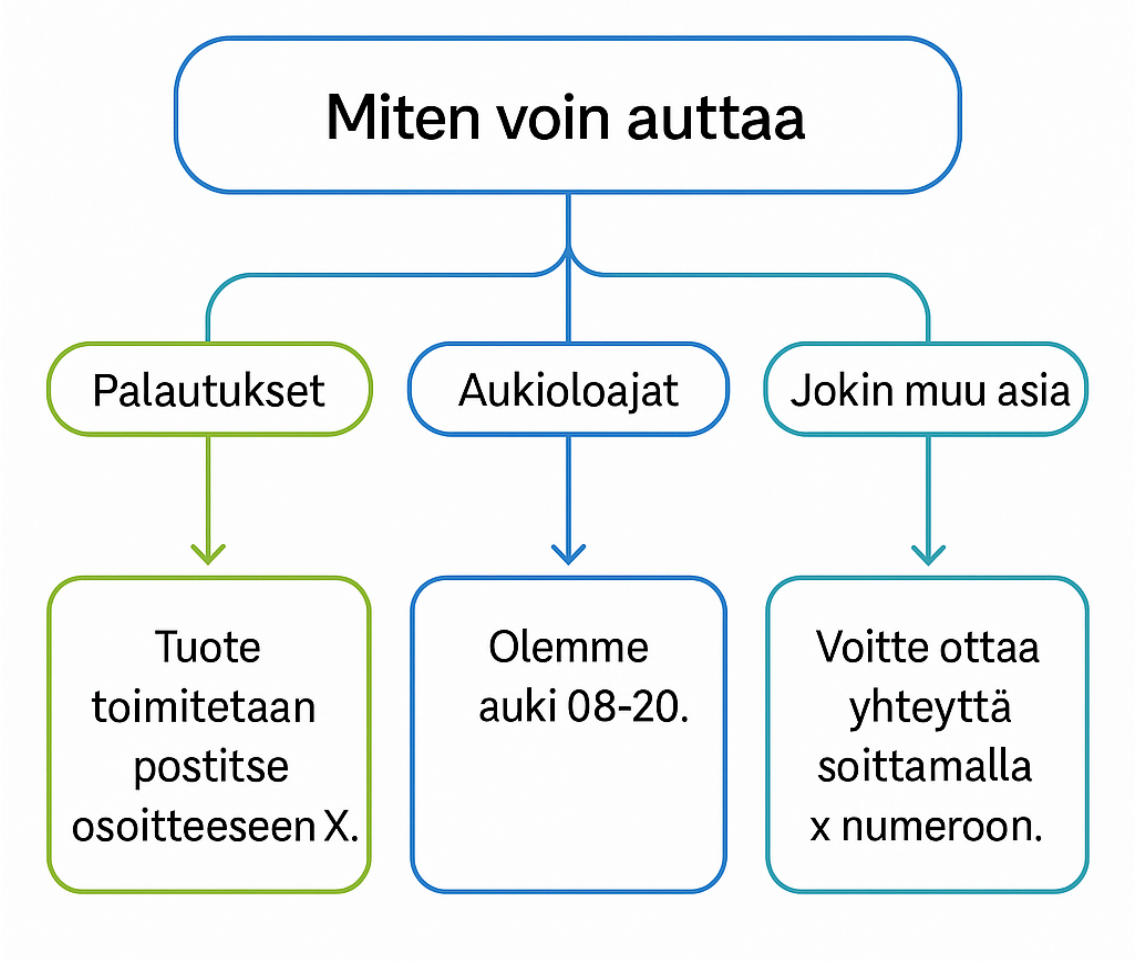
Retrieval-Augmented Generation, jatkossa rag, on menetelmä, jonka tarkoituksena on parantaa generatiivisten kielimallien vastausten tarkkuutta ja luotettavuutta hyödyntämällä asianmukaisia ulkoisia tietolähteitä. RAG-mallissa kielimallin tuottamaa vastausta täydennetään ennen generointia relevantilla, haetulla tiedolla, jolloin vastaukset perustuvat dokumentoituun sisältöön eivätkä pelkästään mallin esikoulutukseen. (Merritt 2025.)

Viime vuosina chatbotit ovat kehittyneet merkittävästi ja pystyvät nykyään tulkitsemaan avoimia kysymyksiä sekä tuottamaan pitkiä ja johdonmukaisia vastauksia. Näitä ratkaisuja hyödynnetään yhä laajemmin tilanteissa, joissa vaaditaan joustavaa vuorovaikutusta ja kykyä käsitellä monimuotoista kielellistä sisältöä. Tällaiset chatbotit pystyvät mukautumaan keskustelutilanteisiin ilman ennalta määriteltyjä vastauspolkuja, mikä tekee niistä huomattavasti joustavampia ja ilmaisukykyisempiä kuin perinteiset sääntöpohjaiset ratkaisut. (Bommasani ym. 2021.)

Sääntöpohjaiset chatbotit perustuvat ennalta määriteltyihin sääntöihin ja valmiisiin vastauspolkuihin. Tällaisissa ratkaisuissa käyttäjän valinnat kulkevat päätöspuumaisen rakenteen läpi, ja mikäli kysymys ei istu valmiisiin kategorioihin, järjestelmä ei pysty tuottamaan täsmällistä vastausta. Tämä rajoittaa niiden käyttöä

erityisesti tilanteissa, joissa käsiteltävä tieto on monimutkaista tai muuttuu nopeasti.

Kuvassa 1 on esitetty tyypillinen sääntöpohjaisen chatbotin rakenne. Käyttäjä valitsee jonkin ennalta määritellyn vaihtoehdon, jonka perusteella botti palauttaa siihen liittyvän vastauksen. Mikäli käyttäjän kysymys ei sovi annettuihin vaihtoehtoihin, järjestelmä ohjaa tämän yleiseen ohjeeseen tai tarjoaa yhteydenottotapaa. Tämä rakenne toimii hyvin yksinkertaisissa ja toistuvissa tilanteissa, mutta se ei skaalaudu laajempiin tai tietointensiivisiin käyttötapauksiin.



KUVA 1. Perinteisen sääntöpohjaisen chatbotin rakenne.

1.1 Tavoitteet ja rajaukset

Opinnäytetyön päätavoitteena oli toteuttaa RAG-arkkitehtuuriin perustuva chatbot, joka hyödyntää organisaation omaan tietopohjaan tallennettua sisältöä ja

tuottaa käyttäjän kysymyksiin mahdollisimman täsmällisiä ja ajantasaisia vastauksia. Toteutettava ratkaisu liitettiin verkkosivupohjaiseen käyttöliittymään, ja sen toimivuutta arvioitiin teknisestä näkökulmasta.

Työn keskeisiä alatavoitteita olivat:

- Suunnitella ja toteuttaa RAG-arkkitehtuuriin perustuva chatbot ja liittää se verkkosivupohjaiseen käyttöliittymään.
- Tutkia, miten RAG vaikuttaa vastausten laatuun, luotettavuuteen ja lähdejäljitettävyyteen.
- Tarkastella ratkaisun suosituskykyä ja vasteaikoja käytännön esimerkkitilanteissa.
- Analysoida, miten ratkaisu skaalautuu eri tietomäärille ja domain kohtaisille aihealueille.

2 TIETOPERUSTA JA KÄSITTEET

2.1 Chatbotit

Chatbotilla tarkoitetaan tietokoneohjelmaa, joka simuloi keskustelua käyttäjän kanssa tekstin tai puheen välityksellä. Chatbotit koetaan tyypillisenä tekoälyjärjestelmän sovelluksena ja yleisenä tietokoneen ja ihmisen vuorovaikutuksen muotona. Chatbotit ovat yleistyneet laajasti eri aloille, kuten markkinointiin, tukipalveluihin, koulutukseen, terveydenhuoltoon, kulttuuriperintöön ja viihteeseen. (Adamopoulou & Moussiades 2020.)

Chatbottien yleistymistä selittävät niiden tarjoamat hyödyt, kuten ympärivuorokautinen saatavuus, nopeat vastaukset ja kyky käsitellä useita keskusteluja ilman jonotusta. Järjestelmät voidaan skaalata palvelemaan kasvavia käyttäjämääriä ilman suoraa henkilöstönlisäystä. Loppukäyttäjän puolella tämä näkyy sujuvampana asiakaskokemuksena ja parempana saavutettavuutena. (Holdsworth s.a.)

Tyypillinen esimerkki on verkkokaupan tukibotti, joka vastaa usein kysyttyihin kysymyksiin toimitusajoista, palautuskäytännöistä ja aukioloajoista. Tämänkaltaiset chatbotit toimivat usein valmiiden sääntöjen ja vastauspolkujen varassa, mikä riittää yksinkertaisiin kysymyksiin mutta rajoittaa monimutkaisempien ja kontekstisidonnaisten kysymysten käsittelyä.

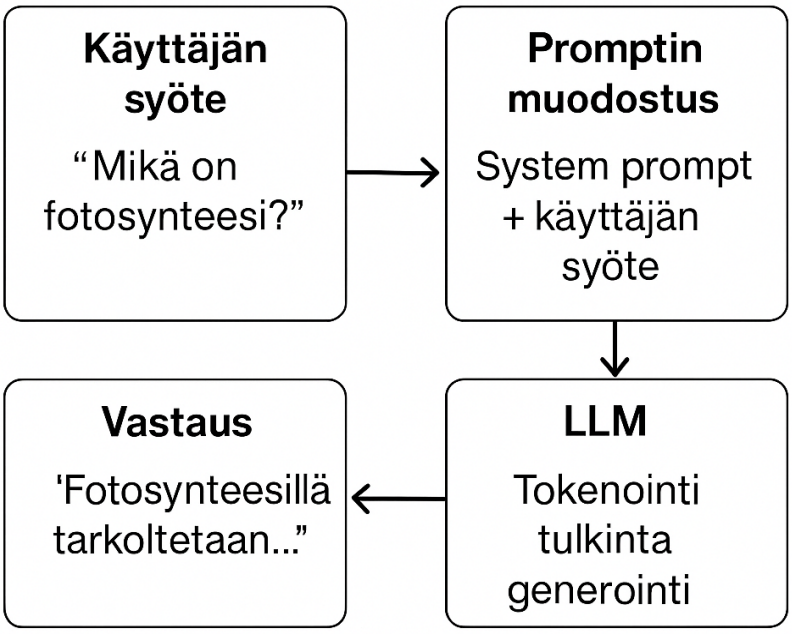
Modernit tekoälypohjaiset chatbotit hyödyntävät luonnollisen kielen käsittelyä ja pystyvät tulkitsemaan käyttäjän tarkoitusta joustavammin. Tämä tarkoittaa, että käyttäjä voi keskustella botin kanssa luonnollisella kielellä eikä sitä ole sidottu ennalta määritettyihin painikkeisiin. Näissä ratkaisuissa vastauksen muodostamisesta vastaa taustalla toimiva kielimalli, eli tekoälymalli, joka on koulutettu tuottamaan ja ymmärtämään luonnollista kieltä suurten tekstiaineistojen perusteella (ks. luku 2.2.2). Tämä tekee vuorovaikutuksesta luonnollisempaa, mutta tuo myös haasteita tietojen luotettavuuden ja ajantasaisuuden kannalta.

Yksi modernien tekoälysovellusten keskeisimpiä komponentteja on prompti. Promptilla tarkoitetaan kielimallille annettavaa ohjetta, joka määrittää sen

tehtävän ja viitekehyksen, jonka perusteella vastaus muodostetaan. Promptit jaetaan tyypillisesti kahteen osaan: system promptiin ja user promptiin. System prompt sisältää mallin toimintaa ohjaavat yleiset säännöt ja roolimäärittelyt, kun taas user prompt vastaa käyttäjän antamaa syötettä, kuten kysymystä tai tehtävänantoa. (Nebuly 2024.)

Käytännössä system prompt voi esimerkiksi määrittää, että botti toimii opettavassa roolissa ja vastaa kysymyksiin asiallisella ja selittävällä tyylillä. User prompt puolestaan sisältää varsinaisen käyttäjän kysymyksen, kuten “Mikä on fotosynteesi?”, johon kielimalli muodostaa vastauksen system promptin asettamien ehtojen mukaisesti.

Kuvassa 2 esitetään yksinkertaistettu chatbotin toimintaketju, jossa käyttäjän antama syöte (user prompt) ja järjestelmän määrittelemä ohjeistus (system prompt) yhdistyvät kokonaisuudeksi, joka välitetään suurelle kielimallille vastauksen generointia varten. Tämä havainnollistaa modernien chatbot-ratkaisujen perusmekanismin ja niiden eron perinteisiin sääntöpohjaisiin järjestelmiin.



KUVA 2. Modernin chatbotin perusdialogivirta ilman RAG-komponentteja.

2.2 Generatiivinen tekoäly ja suuret kielimallit

2.2.1 Generatiivinen tekoäly

Ennen generatiivisen tekoälyn yleistymistä tekoälysovellukset perustuivat malleihin, jotka analysoivat ja luokittelivat valmiita tietoja. Tällaiset mallit arvioivat esimerkiksi röntgenkuvan löydöksiä tai asiakkaan luottokelpoisuutta olemassa olevan datan perusteella. (MIT 2023.)

Generatiivinen tekoäly eroaa perinteisestä mallista tuottamalla kokonaan uutta sisältöä. Se muodostaa tekstiä, kuvia, ääntä tai muuta mediaa oppimiensa rakenteiden ja säännönmukaisuuksien pohjalta, ja tuottaa käyttäjän ohjeiden perusteella luonnollista kieltä muistuttavia vastauksia. (NNLM s.a.). Esimerkiksi käyttäjä voi pyytää selityksen käsitteelle “fotosynteesi” tai pyytää luonnoksen sähköpostista, ja malli tuottaa vastauksen koulutusdatasta oppimansa perusteella.

2.2.2 Suuret kielimallit

Suuret kielimallit (engl. large language model, jatkossa LLM) ovat kehittyneitä generatiivisen tekoälyn sovelluksia. Ne ennustavat seuraavan tokenin aiemman tekstikontekstin perusteella, ja tätä kautta muodostavat johdonmukaisia vastauksia (Brown ym. 2020). Tunnettuja esimerkkejä ovat ChatGPT, Claude ja Gemini.

LLM-mallit oppivat kielen rakenteet ja semanttiset yhteydet valtavien datajoukkojen avulla. Tämän ansiosta samaa mallia voidaan hyödyntää hyvin erilaisiin tehtäviin, kuten tiivistämiseen, kysymysten käsittelyyn ja sisällöntuotantoon, ilman erillistä tehtäväkohtaista koulutusta. (Brown ym. 2020.)

LLM-mallit eivät kuitenkaan automaattisesti tunne organisaation sisäistä tietoa tai tuoreita dokumentteja. Mallin tieto perustuu esikoulutukseen, ja sen tuottama sisältö voi olla vanhentunutta tai vaikeaa jäljittää lähteisiin. Tämä on keskeinen syy hyödyntää menetelmiä, jotka yhdistävät malliin ulkoisen tietolähteen, jolloin malli saa käyttöönsä ajantasaista ja tarkistettavaa tietoa ennen vastauksen muodostamista. (McKinsey & Company 2024.)

2.2.3 Tokenit, konteksti ja kustannus

Tokeni on mallin käyttämä pienin yksikkö. Se voi olla sana, sanan osa tai välimerkki. Malli prosessoi tekstiä tokeni kerrallaan, ja se voi käsitellä vain rajatun määrän tokeneita. Tätä rajoitusta kutsutaan konteksti-ikkunaksi.

Konteksti-ikkunan koko vaikuttaa suoraan:

- Kuinka paljon tietoa malli voi hyödyntää yhtä aikaa
- Kuinka suuria syötteitä se voi käsitellä
- Miten paljon palveluntarjoaja veloittaa käytöstä

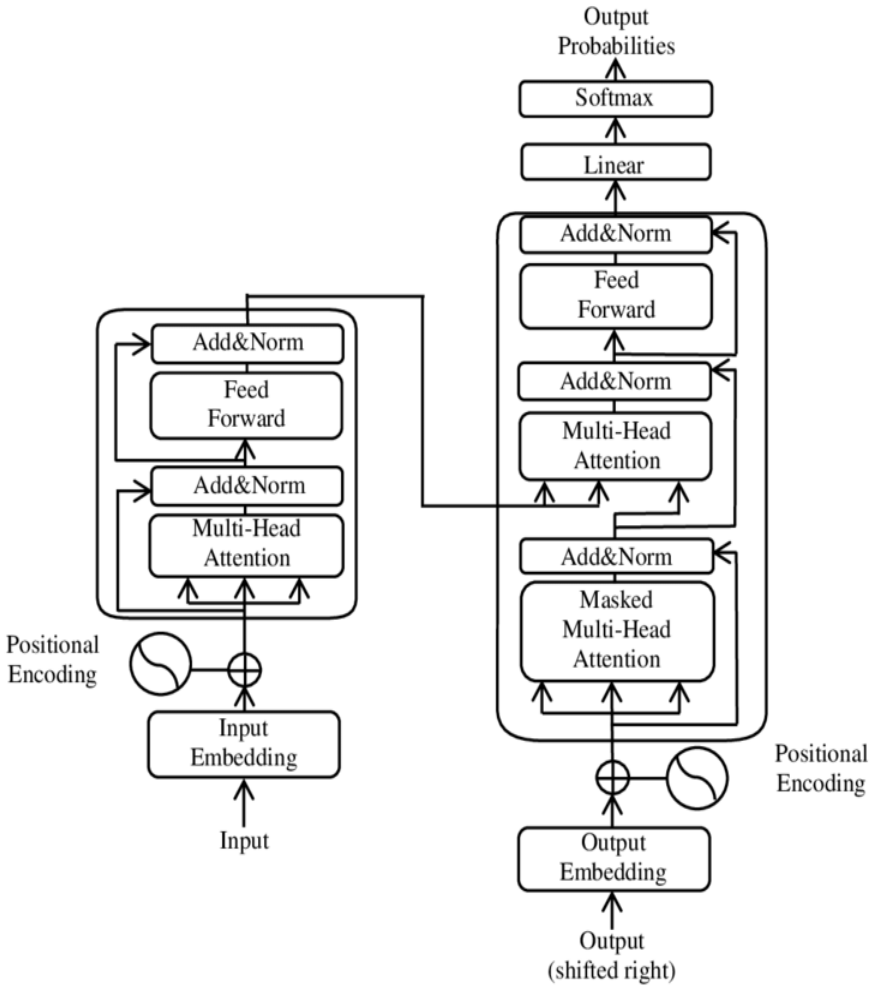
Pitkä konteksti on hyödyllinen, mutta kasvattaa laskentaa ja kustannuksia. Siksi RAG pyrkii tarjoamaan vain olennaiset dokumenttikatkelmat mallille sen sijaan, että sille annetaan koko tietokanta. (OpenAI s.a.; McKinsey & Company 2024.)

2.2.4 Arkkitehtuuri ja vastausten muodostus

Modernit suuret kielimallit perustuvat transformer-arkkitehtuuriin, jonka ytimessä on self-attention-mekanismi. Self-attention laskee jokaiselle tokenille painon suhteessa muihin saman syötteen tokeneihin ja auttaa mallia tunnistamaan, mitkä kohdat tekstistä ovat tärkeimpiä kulloisenkin sanan tai lauseen tulkinnassa. Tämä mahdollistaa pitkien riippuvuuksien mallintamisen ja tehokkaan rinnakkaislaskennan. (Vaswani ym. 2017.)

Teksti käsitellään transformerissa useassa vaiheessa. Ensin syöte pilkotaan tokeneiksi ja muunnetaan vektorimuotoisiksi upotuksiksi (embeddings), jotka kuvaavat sanojen merkitystä numeerisessa muodossa. Koska transformer-arkkitehtuuri ei itsessään huomioi sanajärjestystä, upotuksiin lisätään paikkakoodaus, jonka avulla malli säilyttää tiedon sanojen keskinäisestä järjestyksestä. Tämän jälkeen self-attention- ja syväoppimiskerrosten avulla muodostetaan vektorimuotoinen esitys syötteestä, joka tiivistää tekstin merkitykselliset piirteet ja toimii pohjana vastauksen generoinnille. (Vaswani ym. 2017; Lewis ym. 2020.)

Kuvassa 3 esitetään transformer-mallin perusrakenne encoder–decoder-jaotteluineen. Encoder tuottaa syötteestä kontekstuaalisen vektoriedustuksen, ja decoder muodostaa vastauksen token kerrallaan hyödyntäen sekä encoderin tuottamaa esitystä että aiemmin generoituja tokeneita. Näin malli tuottaa kielellisesti sujuvan vastauksen, joka perustuu tilastolliseen ennustamiseen ja opittuihin kielellisiin rakenteisiin.



KUVA 3. Transformer-mallin perusrakenne.

2.2.5 Hallusinaatiot

Hallusinaatio tarkoittaa tilannetta, jossa malli tuottaa sujuvaa mutta virheellistä tai keksittyä sisältöä. Ilmiö johtuu mallin tilastollisesta luonteesta: jos konteksti ei sisällä riittävästi faktatietoa tai malli ei ole oppinut aiheeseen liittyvää tietoa koulutuksessaan, malli täydentää vastauksen todennäköisimmän jatkon perusteella.

Käytännössä hallusinaatiot voivat ilmetä esimerkiksi siten, että malli esittää uskottavalta kuulostavia mutta todellisuudessa virheellisiä faktoja tai lähteitä. OpenAI huomauttaa, että tavanomaiset arviointikäytännöt palkitsevat usein arvailua enemmän kuin epävarmuuden rehellistä ilmaisemista, mikä ylläpitää hallusinaatioita erityisesti tietointensiivisissä tehtävissä (OpenAI s.a.).

3 RETRIEVAL-AUGMENTED GENERATION

Suuren kielimallin tekemät vastaukset perustuvat aina siihen tietoon, joka sille annetaan. Malli muodostaa vastauksensa käyttäjän antaman ohjeen eli promptin perusteella. Promptin muotoilu, kuten kysymyksen asettelu, tehtävänanto tai kehystys vaikuttaa suoraan siihen, millaisen vastauksen malli tuottaa. Tieteelliset tutkimukset osoittavat, että pienetkin muutokset promptissa voivat parantaa tai heikentää suorituskykyä merkittävästi, ja eri promptirakenteet tuottavat systemaattisesti erilaista laatua. (Gozzi & Di Maio 2024; Brucks ym. 2025.)

Kielimallin suoriutuminen riippuu myös siitä, millaista kontekstia sen käyttöön annetaan. Jos mallille ei tarjota relevanttia tietoa, se täydentää puuttuvat kohdat omilla tilastollisilla arvauksillaan, jolloin virheiden ja hallusinaatioiden riski kasvaa. (Huyen 2024.)

Retrieval-Augmented Generation (RAG) ratkaisee tämän ongelman yhdistämällä suuren kielimallin ulkoiseen tietolähteeseen. Ennen vastauksen muodostamista järjestelmä hakee käyttäjän kysymykseen liittyviä tekstikatkelmia ja liittää ne osaksi promptia, jolloin malli saa käyttöönsä ajantasaisen ja tarkistettavissa olevan kontekstin (McKinsey & Company 2024). Näin RAG tuottaa vastauksia, jotka perustuvat dokumentoituun tietoon eivätkä mallin omiin arvauksiin.

RAG on erityisen hyödyllinen yritys- ja toimialakohtaisissa sovelluksissa, joissa tarvitaan täsmällistä, lähdejäljitettävää ja ajantasaista tietoa. Sen etuna on se, että yrityksen omat dokumentit voidaan hyödyntää ilman raskasta mallin uudelleenkoulutusta.

RAG-järjestelmät rakentuvat tyypillisesti kolmesta päävaiheesta:

- Tiedon tallennus ja indeksointi: lähteiden pilkkominen ja muuntaminen vektoreiksi.
- Haku: relevanttien tekstikatkelmien haku semanttisesti.
- Generointi: vastaus tuotetaan yhdistämällä haettu konteksti ja käyttäjän kysymys.

3.1 Data ingestion ja esikäsittely

Data ingestion tarkoittaa datan keräämistä, tallentamista ja siirtämistä järjestelmän käsiteltäväksi. RAG-järjestelmissä ingestion sisältää esimerkiksi dokumenttien tuonnin eri lähteistä (PDF-tiedostot, verkkosivut, tietokannat) ja niiden muuntamisen yhtenäiseen tekstimuotoon. Tämän yhteydessä voidaan puhdistaa turhat elementit, kuten navigaatio- ja mainospalkit. (AWS s.a.)

Esikäsittelyyn kuuluu lisäksi metatiedon, kuten otsikoiden, aikaleimojen ja kategoriatietojen lisääminen. Nämä tiedot voidaan tallentaa yhdessä tekstin kanssa ja hyödyntää myöhemmin haussa, esimerkiksi rajaamalla tuloksia tiettyyn aikajaksoon tai vain tietyn osaston tuottamiin dokumentteihin. Hyvin suunniteltu ingestion-vaihe helpottaa myöhempiä vaiheita ja parantaa sekä haun että generoinnin laatua. (Bronsdon 2025.)

3.2 Chunking

Chunking tarkoittaa dokumenttien jakamista pienempiin osiin, kuten kappaleisiin tai lausekokonaisuuksiin. Liian suuret tekstikatkelmat sisältävät helposti liikaa epäolennaista sisältöä, jolloin haku tuo mukaan tarpeetonta tietoa. Liian pienet tekstikatkelmat taas katkaisevat merkityksiä ja vaikeuttavat kokonaisuuksien ymmärtämistä. Optimaalinen tekstikatkelman koko on tyypillisesti 200–500 tokenia, mutta sopiva arvo riippuu aineiston rakenteesta ja käyttötapauksesta. (Bhat ym. 2025.)

Käytännössä tekstikappaleiden pilkkominen voi tarkoittaa esimerkiksi sitä, että jokainen intranet-ohjeen kappale muodostaa oman kappaleensa. Tällöin chatbot voi hakea tiettyä käyttötilannetta käsittelevän kappaleen ja käyttää sitä suoraan vastauksen pohjana sen sijaan, että koko ohje luettaisiin kerralla.

3.3 Datan vektorointi ja embeddings

Embedding (suom. vektoriesitys / upotus) on numeerinen esitys tekstistä. Embeddingien avulla semanttisesti samankaltaiset tekstit sijoittuvat lähelle toisiaan

vektoriavaruudessa. RAG-järjestelmissä jokaisesta tekstikatkelmasta lasketaan embedding, joka tallennetaan vektorikantaan yhdessä alkuperäisen tekstin ja metatiedon kanssa. (Ali & Khan 2024.)

Embedding-mallin valinta vaikuttaa siihen, miten hyvin haku löytää merkityksellään oikeat katkelmat. Esimerkiksi yleiseen tekstikieleen koulutettu embedding-malli toimii hyvin, kun käsitellään luonnollista kieltä, mutta erityisalojen tekniset termit voivat edellyttää erikseen hienosäädettyjä tai alakohtaisia malleja. (OpenAI s.a.)

3.4 Vektorihaku

Vektorihaku on hakutekniikka, jossa sana, dokumentti tai kysymys muunnetaan vektoreiksi, ja haussa etsitään semanttisesti samankaltaisia vektoreita korrelaatioiden perusteella (Meredith & Russi 2025). Menetelmä mahdollistaa merkityssisällön tunnistamisen pelkkien hakusanojen sijaan, mikä on keskeistä, kun chatbot hakee tietoa yrityksen sisäisestä tietopohjasta. Tuloksena saadaan joukko relevantteja katkelmia, jotka muodostavat kielimallin käytettävän kontekstin. (McKinsey & Company 2024.)

McKinseyn (2024) mukaan retrieval on RAG-prosessin keskeinen vaihe, sillä sen kautta malli saa käyttöönsä ajantasaisen tiedon ilman tarvetta mallin uudelleenkoulutukselle. Haun tarkoitus ei ole vain löytää tietoa, vaan rajata ja rikastaa sitä niin, että generointivaiheessa mallilla on riittävä mutta hallittu määrä kontekstia.

SIGIR'24-konferenssissa julkaistu tutkimus The Power of Noise: Redefining Retrieval for RAG Systems tarkasteli hakukerroksen laatua ja sen vaikutusta generointiin. Tulokset osoittavat, että liian suuri tai heikkolaatuinen hakukonteksti heikentää vastausten tarkkuutta, kun taas kohinaa vähentämällä ja tuloksia uudelleenjärjestämällä voidaan parantaa generointia merkittävästi (Cuconasu ym. 2024). Tutkimuksen keskeinen havainto oli, että RAG ei ole "mitä enemmän, sen parempi" -järjestelmä, vaan laadun ja määrän tasapaino on ratkaisevaa.

Myös EMNLP'24:n Summary of a Haystack -tutkimus tukee tätä näkemystä. Siinä havaittiin, että pitkänkään kontekstin LLM ei suoriudu tehokkaasti ilman

hakukomponenttia, joka poimii tehtävään olennaisen tiedon (Laban ym. 2024). RAG-järjestelmien hakuvaihe siis määrittää pitkälti koko mallin tietopohjaisen suorituskyvyn ja vastauksen luotettavuuden (McKinsey & Company 2024; Cuconasu ym. 2024; Laban ym. 2024).

3.5 LLM ja vastausten muodostus

Generointivaiheessa LLM yhdistää käyttäjän kysymyksen ja tietopankista haetut dokumenttikatkelmat. Malli tuottaa vastauksen vain annetun kontekstin perusteella, mikä vähentää hallusinaatioita ja parantaa vastausten läpinäkyvyyttä. Käyttäjälle tämä näkyy esimerkiksi siten, että vastauksen yhteydessä voidaan esittää myös käytettyjen lähteiden otsikot tai linkit. (McKinsey & Company 2024.)

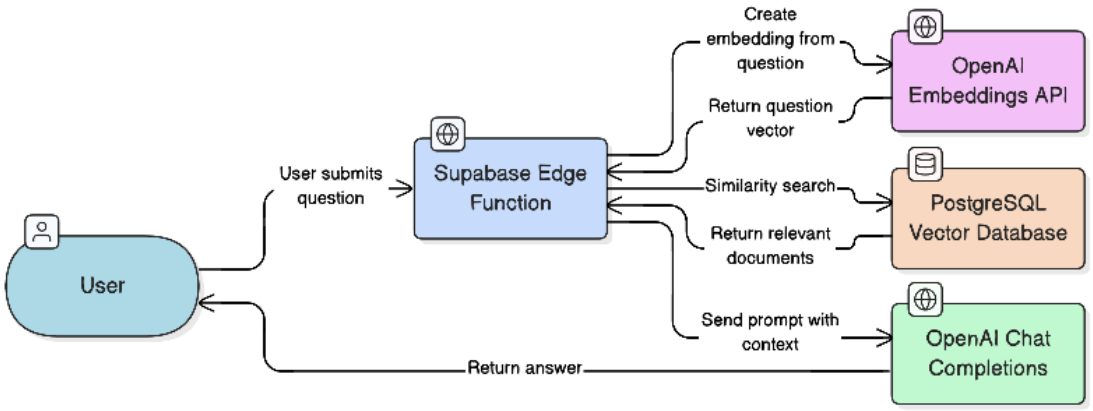
Käytännössä prosessi etenee niin, että järjestelmä rakentaa promptin, joka sisältää sekä käyttäjän alkuperäisen kysymyksen että haetut tekstikatkelmat. Tämä prompti syötetään LLM:lle, joka laatii lopullisen vastauksen. Jos retrieval-vaihe on onnistunut ja konteksti laadukas, vastaus on sekä kielellisesti sujuva että tiedollisesti oikeaan aineistoon sidottu.

4 TOTEUTUS

4.1 Toteutuksen teknologiat ja arkkitehtuuri

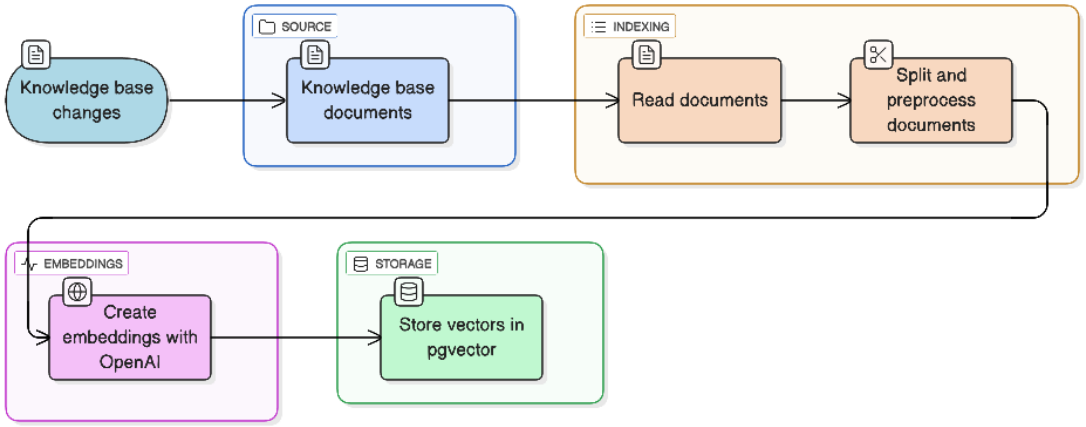
Toteutettu ratkaisu rakentuu kolmesta pääkomponentista: verkkosivulle upotetusta chat-käyttöliittymästä, Supabasen tarjoamista taustapalveluista sekä PostgreSQL-tietokannasta, jota on laajennettu pgvector-moduulilla. Supabase on avoimen lähdekoodin Backend-as-a-Service (BaaS) -alusta, joka tarjoaa sovelluksille valmiita taustapalveluja, kuten tietokannan, rajapinnat ja palvelinpuolen logiikan hallinnan (Kosisochukwu 2025). Pgvector taas on erityisen hyödyllinen sovelluksissa, joissa hyödynnetään suuria kielimalleja, sillä se mahdollistaa tekstisisällön vektorimuotoisten esitysten tallentamisen ja semanttisen haun suoraan relaatiotietokannassa (Supabase s.a). Tämän ansiosta vektorihaku voitiin toteuttaa ilman erillistä vektoritietokantapalvelua hyödyntäen jo käytössä olevia palveluita.

Kuvassa 4 esitetään järjestelmän korkean tason arkkitehtuurin online-puoli. Kuva havainnollistaa, miten käyttäjän tekemä kysely etenee järjestelmän eri komponenttien välillä aina vastauksen muodostamiseen saakka. Online-arkkitehtuuri kattaa käyttäjän ja järjestelmän välisen vuorovaikutuksen sekä ne keskeiset palvelut, joita hyödynnetään yksittäisen kyselyn, eli viestin, käsittelyssä.



KUVA 4. Online-vaiheen korkean tason arkkitehtuurikaavio.

Tietopankin sisältö käsitellään järjestelmässä erillisessä, manuaalisesti suoritettavassa offline-prosessissa, joka esitetään kuvassa 5. offline-arkkitehtuurin tehtävänä on valmistella tietopankin sisältö sellaiseen muotoon, että sitä voidaan hyödyntää tehokkaasti käyttäjäkyselyiden käsittelyssä. Prosessin aikana dokumentit luetaan lähdejärjestelmästä, pilkotaan pienempiin osiin ja muunnetaan vektorimuotoisiksi esityksiksi. Muodostetut vektorit tallennetaan tietokantaan, jossa ne ovat myöhemmin hyödynnettävissä semanttisessa haussa. Varsinainen käyttäjäkyselyihin liittyvä online-prosessi hyödyntää näitä valmiiksi tallennettuja vektoreita, mikä mahdollistaa järjestelmän nopean vasteajan ja selkeän erottelun online- ja offline-vaiheiden välillä.



KUVA 5. Offline-vaiheen korkean tason arkkitehtuurikaavio.

4.2 Tietokantakerros

4.2.1 Pgvector-laajennus ja käyttöönotto

Tietokantakerros muodostaa järjestelmän rakenteellisen perustan, sillä semanttinen haku ja kielimallin hyödyntäminen edellyttävät mahdollisuutta tallentaa ja vertailla tekstisisältöä merkityksen perusteella. Ratkaisu toteutettiin hyödyntämällä jo valmiiksi käytössä olevaa PostgreSQL-tietokantaa, jota laajennettiin pgvector-moduulilla. Laajennus otettiin käyttöön tietokantamigraatiossa, mikä mahdollistaa vektorimuotoisen datan käsittelyn suoraan relaatiotietokannassa.

Pgvector-laajennuksen käyttöönotto sekä tietopankin perusrakenteen määrittely on esitetty kuvassa 6. Migraatiossa aktivoidaan ensin pgvector-laajennus, minkä

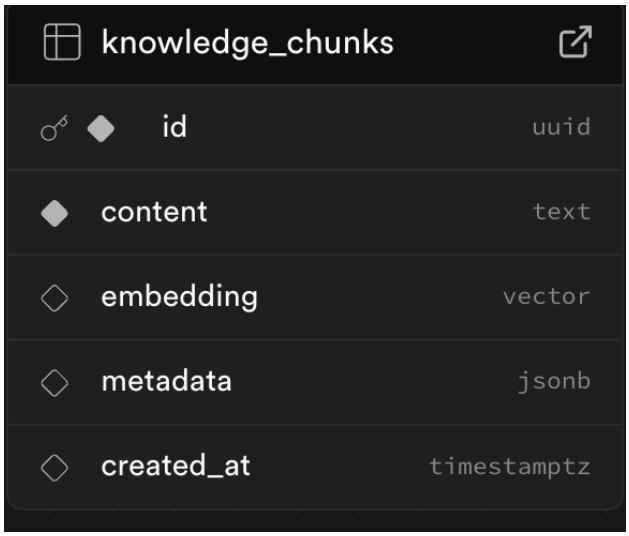
jälkeen luodaan tietopankin sisältöä varten taulu. Tämä vaihe luo edellytykset vektorien tallentamiselle ja myöhemmälle semanttiselle haulle.

```
1   -- Enable pgvector extension for vector similarity search
2   CREATE EXTENSION IF NOT EXISTS vector;
3
4   -- Knowledge chunks table for RAG
5   -- Stores document chunks with their embeddings for semantic search
6   CREATE TABLE IF NOT EXISTS knowledge_chunks (
7     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
8     content TEXT NOT NULL,
9     embedding vector(1536), -- OpenAI text-embedding-3-small dimension
10    metadata JSONB DEFAULT '{}',
11    created_at TIMESTAMPTZ DEFAULT now()
12  );
13
```

KUVA 6. Pgvector-laajennuksen käyttöönotto ja knowledge_chunks-taulun luonti.

4.2.2 Tietopankin rakenne

Tietopankin sisältö tallennettiin knowledge_chunks-tauluun, jossa jokainen rivi vastaa yhtä rajattua tekstikatkelmaa. Taulun rakenne on esitetty kuvassa 7. Taulu sisältää yksilöllisen UUID-muotoisen tunnisteen, varsinaisen tekstisisällön, vektoriesityksen, metatiedot sekä aikaleiman. Vektoriesityksen ulottuvuus on 1536, mikä vastaa käytetyn embedding-mallin tuottamaa vektorin pituutta.



KUVA 7. Knowledge_chunks-taulun rakenne PostgreSQL-tietokannassa.

4.2.3 Semanttinen haku tietokantafunktion avulla

Semanttista hakua varten tietokantaan määriteltiin match_knowledge-niminen funktio sekä vektorihakua tehostava indeksi. Näiden määrittelyt on esitetty kuvassa 8. Funktio vastaanottaa käyttäjäkysymyksen vektoriesityksen ja palauttaa merkitykseltään käyttäjäkysymystä lähimmät tekstikatkelmat. Hakutulokset järjestetään samankaltaisuuden mukaan, ja palautettavien tulosten määrää voidaan rajoittaa parametrien avulla.

```sql
-- Function to find similar knowledge chunks
-- Uses cosine similarity (1 - cosine distance)
CREATE OR REPLACE FUNCTION match_knowledge(
  query_embedding vector(1536),
  match_count int DEFAULT 5,
  match_threshold float DEFAULT 0.5
)
RETURNS TABLE (
  id UUID,
  content TEXT,
  metadata JSONB,
  similarity FLOAT
)
LANGUAGE plpgsql
AS $$
BEGIN
  RETURN QUERY
  SELECT
    kc.id,
    kc.content,
    kc.metadata,
    1 - (kc.embedding <=> query_embedding) AS similarity
  FROM knowledge_chunks kc
  WHERE 1 - (kc.embedding <=> query_embedding) > match_threshold
  ORDER BY kc.embedding <=> query_embedding
  LIMIT match_count;
END;
$$;
```

KUVA 8. Match_knowledge funktio.

4.2.4 Hakusuorituskyvyn optimointi vektori-indeksillä

Hakulogiikka sijoitettiin tietokantafunktion sisään. Ratkaisu vähentää taustapalveluiden ja tietokannan välistä tiedonsiirtoa sekä siirtää laskennan lähemmäs

varsinaista dataa, mikä on yleinen suorituskykyä parantava periaate relaatiotietokantojen suunnittelussa (PostgreSQL Global Development Group, s.a.).

Hakusuorituskykyä parannettiin luomalla vektorikentälle Hierarchical Navigable Small World (HNSW) -indeksi, joka soveltuu tehokkaaseen vektoripohjaiseen hakuun. Indeksi perustuu vektoreista muodostettuun graafirakenteeseen, jonka avulla lähimpien naapureiden haku voidaan toteuttaa tehokkaasti ilman täydellistä läpikäyntiä. Menetelmä mahdollistaa huomattavasti nopeammat vasteajat erityisesti silloin, kun tietopankin koko kasvaa. (Malkov & Yashunin, 2018). Indeksin luonti tietokantaan on esitetty kuvassa 9.

```
14  -- Index for fast similarity search using HNSW algorithm
15  -- HNSW provides good recall with fast query times
16  CREATE INDEX IF NOT EXISTS knowledge_chunks_embedding_idx
17  ON knowledge_chunks
18  USING hnsw (embedding vector_cosine_ops);
19
```

KUVA 9. HNSW indexin luonti.

4.2.5 Tietoturva ja käyttäjäoikeuksien hallinta

Tietoturvaa vahvistettiin ottamalla käyttöön rivitason käyttöoikeudet (Row Level Security, RLS), joilla tietokantataulun käyttöä voidaan rajata kyselykohtaisesti. RLS:n avulla määriteltiin erilliset käytännöt tietopankin lukemiselle ja muokkaamiselle.

Tietopankin sisällön lukeminen sallittiin anonyymeille ja autentikoiduille käyttäjille. Tämä oli tarpeen, koska chatbot on tarkoitettu käytettäväksi ilman käyttäjäkohtaista autentikointia ja sen tulee pystyä hakemaan tietoa tietopankista kaikkien sivuston käyttäjien puolesta. Lukuoikeus ei kuitenkaan tarkoita suoraa pääsyä tietokantaan, vaan haku tapahtuu palvelinpuolen logiikan kautta rajatun toiminnallisuuden välityksellä.

Tietopankin sisällön lisääminen, päivittäminen ja poistaminen rajattiin ainoastaan palvelinpuolen roolille. Näin varmistetaan, ettei loppukäyttäjä voi muokata tietokannan sisältöä tai vaikuttaa tietopankin rakenteeseen. RLS-käytäntöjen

määrittely tietokantamigraatiossa on esitetty kuvassa 10.

```
49  -- Allow public access to the match function (for edge functions)
50  GRANT EXECUTE ON FUNCTION match_knowledge TO anon, authenticated;
51
52  -- RLS: Allow read access to all, write only to authenticated service role
53  ALTER TABLE knowledge_chunks ENABLE ROW LEVEL SECURITY;
54
55  CREATE POLICY "Allow public read access to knowledge chunks"
56  ON knowledge_chunks FOR SELECT
57  TO anon, authenticated
58  USING (true);
59
60  CREATE POLICY "Allow service role to insert knowledge chunks"
61  ON knowledge_chunks FOR INSERT
62  TO service_role
63  WITH CHECK (true);
64
65  CREATE POLICY "Allow service role to update knowledge chunks"
66  ON knowledge_chunks FOR UPDATE
67  TO service_role
68  USING (true);
69
70  CREATE POLICY "Allow service role to delete knowledge chunks"
71  ON knowledge_chunks FOR DELETE
72  TO service_role
73  USING (true);
74
```

KUVA 10. RLS-policy määrittelyt.

4.3 Kysely-vastausprosessi

4.3.1 Relevantin kontekstin haku tietokannasta

Kysymyksen vektoroinnin jälkeen järjestelmä hakee tietokannasta semanttisesti lähimmät tekstikatkelmat. Haku toteutettiin kutsumalla tietokantaan määriteltyä match_knowledge-funktiota, joka palauttaa kysymyksen vektoriesitystä lähimpänä olevat tietopankin osiot.

Palautettavien tulosten määräksi määritettiin kolme. Valinta muodostaa tasapainon kontekstin kattavuuden ja selkeyden välillä. Liian laaja konteksti kasvattaisi promptin kokoa ja voisi heikentää vastausten kohdistuvuutta, kun taas liian suppea konteksti saattaisi jättää olennaista tietoa pois.

4.3.2 Promptin muodostaminen tietopankkikontekstilla

Haetut tekstikatkelmat liitettiin osaksi järjestelmäpromptia, jolla ohjataan kielimallin toimintaa. Prompti koostuu chatbotin roolin ja toimintaperiaatteiden määrittelystä, tietopankista haetusta kontekstista sekä vastaustyyliä koskevista ohjeista. Ohjeistuksessa korostettiin seuraavia seikkoja:

- Asiakaspalveluhenkistä ja ammattimaista sävyä.
- Käyttäjän kielen huomioimista.
- Vastausten rajaamista ainoastaan tietopankista saatavissa olevaan sisältöön.

Mikäli tietopankista ei löytynyt riittävän relevanttia sisältöä, promptiin liitettiin erillinen ohje, jonka perusteella kielimallia ohjattiin välttämään spekulointia ja ohjaamaan käyttäjä tarvittaessa ottamaan yhteyttä asiakaspalveluun. Tällä ratkaisulla pyrittiin vähentämään virheellisten tai keksittyjen vastausten syntymistä tilanteissa, joissa tietopankki ei sisällä vastausta käyttäjän kysymykseen.

Promptin ohjauksessa hyödynnettiin few-shot prompting -menetelmää, joka esitetään kuvassa 11. Menetelmässä kielimallille annetaan yksi tai useampi esimerkkikysymys ja -vastaus, joiden avulla mallille havainnollistetaan toivottua vastaustapaa. Esimerkkien avulla kielimalli kykenee noudattamaan annettuja ohjeita johdonmukaisemmin kuin pelkän sanallisen ohjeistuksen perusteella. Few-shot prompting on vakiintunut tapa parantaa suurten kielimallien ohjeidenmukaista käyttäytymistä erityisesti rajatuissa tehtävissä (Brown ym. 2020).

```
119  Kysymys:
120  "Miten SkillResume auttaa ATS-optimoinnissa?"
121
122  Hyvä vastaus:
123  "SkillResume auttaa ATS-optimoinnissa seuraavasti:
124
125  1. Rakenteet ja otsikot on suunniteltu ATS-järjestelmiä varten
126  2. Avainsanat sijoitetaan oikein eri osioihin
127  3. CV:n muotoilu pysyy selkeänä ja koneellisesti luettavana"
128
```

KUVA 11. Few-Shot prompti.

4.3.3 Vastauksen generointi ja keskusteluhistorian hyödyntäminen

Viimeisessä vaiheessa kielimalli generoi vastauksen järjestelmäpromptin, käyttäjän kysymyksen sekä rajatun keskusteluhistorian perusteella. Keskusteluhistoriaa hyödynnettiin, jotta chatbot kykenee vastaamaan luontevasti jatkokysymyksiin ja viittaamaan aiempiin viesteihin. Historian pituus rajattiin kuuteen viimeisimpään viestiin, mikä auttaa hallitsemaan kustannuksia ja pitää pyynnön koon kohtuullisena.

Kielimalliksi valittiin gpt-4o-mini, joka tarjoaa hyvän tasapainon vasteajan, kustannusten ja vastausten laadun välillä. Vastauksen generointia ohjattiin lämpötilaparametrilla sekä maksimipituutta rajaavalla token-arvolla, joiden avulla vastaukset pysyvät johdonmukaisina ja riittävän tiiviinä. Vastauksen generointikutsu on esitetty kuvassa 12.

```
/**
 * Generate chat response using the LLM
 */
async function generateResponse(
  systemPrompt: string,
  userMessage: string,
  history: ChatMessage[]
): Promise<string> {
  const messages: ChatMessage[] = [
    { role: "system", content: systemPrompt },
    ...history.slice(-6), // Keep last 6 messages for context
    { role: "user", content: userMessage },
  ];

  const response = await fetch("https://api.openai.com/v1/chat/completions", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${OPENAI_API_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      model: CHAT_MODEL,
      messages,
      temperature: 0.7,
      max_tokens: 500,
    }),
  });

  if (!response.ok) {
    const error = await response.text();
    throw new Error(`Chat API error: ${error}`);
  }

  const data = await response.json();
  return data.choices[0].message.content;
}
```

KUVA 12. Vastauksen generointi ja parametrit.

4.3.4 Prosessin ohjausparametrit

Vastausprosessin toimintaan vaikuttavat keskeisesti kielimallin valinta, generointiparametrit sekä tietopankista haettavien tekstikatkelmien määrä. Näiden arvojen säätäminen vaikuttaa suoraan vastausten laatuun, pituuteen ja kustannuksiin. Tässä toteutuksessa parametrit määritettiin siten, että vastaukset pysyvät napakoina ja tietopankin sisältöä hyödynnetään tehokkaasti ilman tarpeetonta kontekstin paisumista. Valitut parametrit, sekä valintaperusteet, esitetään kuvassa 13.

Parametri	Arvo	Vaikutus
model	gpt-4o-mini	Kustannustehokas ja nopea
temperature	0.7	Tasapaino johdonmukaisuuden ja luonnollisuuden välillä
max_tokens	500	Estää liian pitkät vastaukset ja kontrolloi hintaa
TOP_K_CHUNKS	3	Kontekstin laajuus

KUVA 13. Määritettävät parametrit.

4.4 Tietopankin hallinta

Tietopankin päivittäminen on keskeinen ylläpitotehtävä RAG-järjestelmässä, koska chatbotin vastausten tulee perustua ajantasaiseen sisältöön. Kun palvelun ominaisuudet, hinnat tai toimintatavat muuttuvat, tietopankki päivitetään vastaamaan uusia tietoja.

Tietopankin sisältöä päivitetään hallitusti erillisen latausskriptin avulla, joka esitetään kuvassa 14. Ratkaisu perustuu siihen, että tietopankki ylläpidetään tavallisena tekstitiedostona (knowledge-base.txt), jota muokataan manuaalisesti tarpeen mukaan. Tämä lähestymistapa valittiin sen selkeyden ja helppokäyttöisyyden vuoksi: sisältöä voidaan päivittää ilman erillisiä hallintatyökaluja tai teknistä käyttöliittymää.

```
4. Mikä tarkoittaa “ATS-optimoitu CV”?

ATS-optimoitu CV tarkoittaa ansioluetteloa, jonka:

Rakenteen ATS osaa lukea oikein

Osiot ovat selkeitä (Työkokemus, Koulutus, Taidot jne.)

Teksti on pelkistetyssä muodossa

Ei ole taulukoita, grafiikkaa, sarakkeita tai kuvia

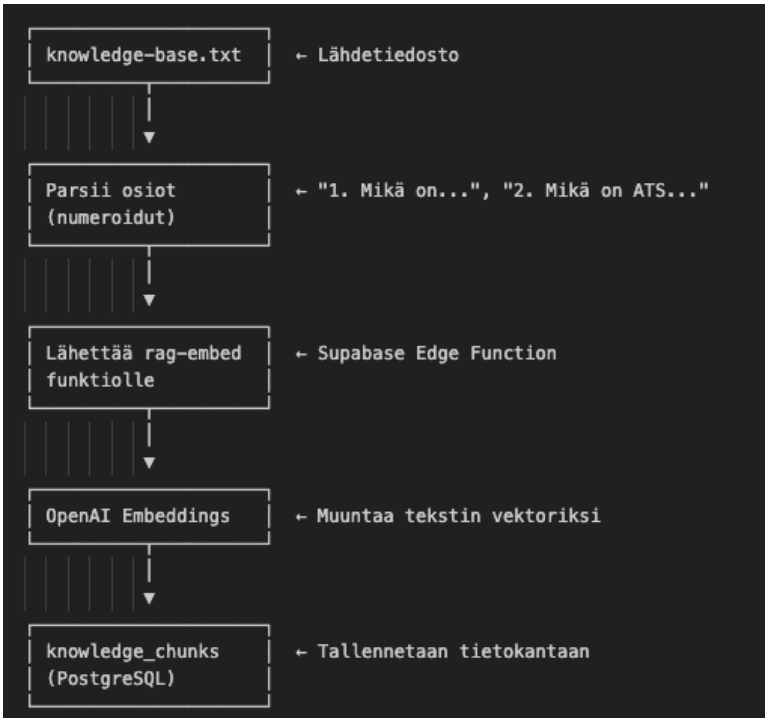
Ei käytetä erikoisia fontteja tai symboleita

Sisältää työpaikkailmoitukseen liittyviä olennaisia avainsanoja

SkillResume käyttää juuri näitä periaatteita kaikissa pohjissaan.
```

KUVA 14. Tietopankin jäsentely tekstitiedostossa.

Kun sisältö on jäsennelty, osiot lähetetään yksi kerrallaan vektorointipalvelulle, joka huolehtii tekstin pilkkomisesta, embeddingien muodostamisesta ja tallennuksesta tietokantaan. Tietopankin päivitys suoritetaan tarvittaessa tyhjentämällä vanha sisältö ja lataamalla päivitetty sisältö uudelleen latausskriptin avulla. Tiedon kulku tietokantaan ja vektoreiksi on esitetty kaaviona kuvassa 15.

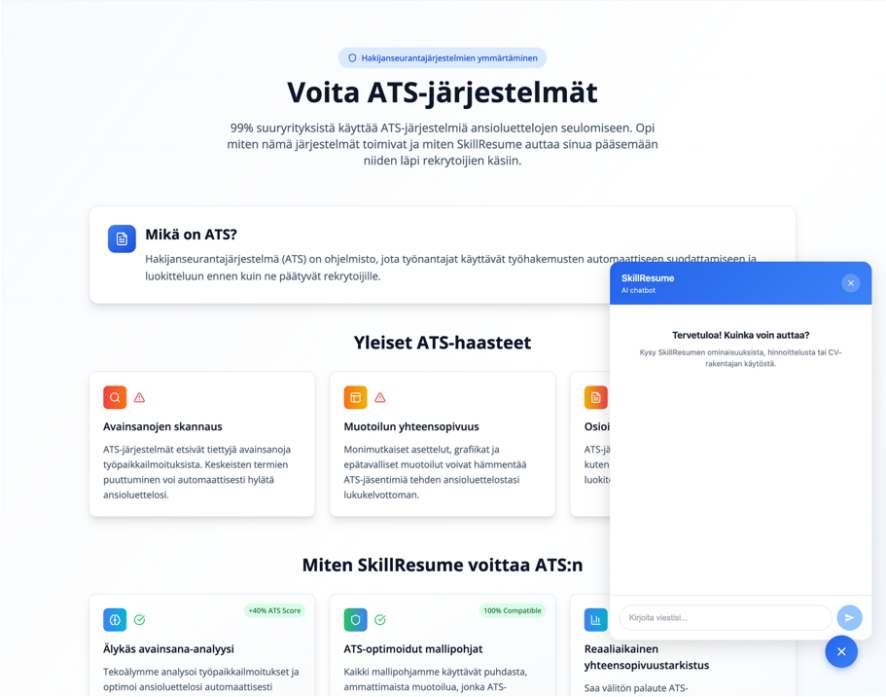


KUVA 15. Tiedonkulku kaaviona txt-tiedostosta vektoreiksi ja tietopankkiin.

4.5 Käyttöliittymä

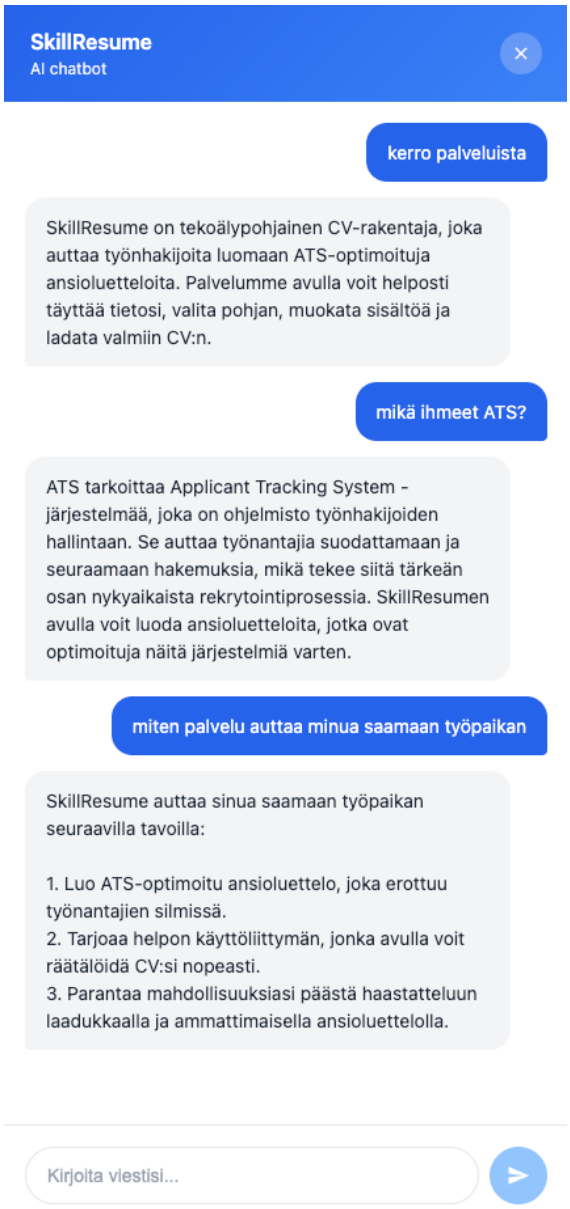
Chatbotin käyttöliittymä toteutettiin verkkosivulle upotettavana vempaimena, jonka tavoitteena on tarjota käyttäjälle helppo tapa esittää palveluun liittyviä kysymyksiä. Käyttöliittymä on suunniteltu siten, että se voidaan liittää osaksi olemassa olevaa verkkosivua ilman, että sivuston muut tyylit tai rakenteet vaikuttavat sen toimintaan.

Käyttöliittymä esitetään käyttäjälle kelluvana chat-kuplana, josta avautuu varsinainen keskusteluikkuna. Työpöytälaitteilla keskusteluikkuna näkyy näytön oikeassa alakulmassa, kun taas mobiililaitteilla se avautuu koko näytön kokoisena näkymänä. Tämä asettelu parantaa käytettävyyttä eri päätelaitteilla ilman tarvetta erillisille käyttöliittymäversioille. Chatbotin käyttöliittymä osana verkkosivua on esitetty kuvassa 16.



KUVA 16. Ison näytön käyttöliittymä.

Käyttöliittymän ulkoasu ja toiminta on suunniteltu siten, että käyttäjä ohjautuu luontevasti esittämään kysymyksiä palvelun ominaisuuksista ja käytöstä. Mobiililaitteilla koko näytön tilassa avautuva keskusteluikkuna helpottaa lukemista ja viestien kirjoittamista, mikä on havainnollistettu kuvassa 17.



KUVA 17. Mobiililaitteen käyttöliittymä.

Kehitystyössä kiinnitettiin erityistä huomiota siihen, että käyttöliittymä toimii yhdenmukaisesti eri sivuilla ja eri ympäristöissä. Tämän ansiosta chatbot voidaan lisätä osaksi palvelua ilman, että se edellyttää muutoksia sivuston olemassa olevaan rakenteeseen tai tyyliasetuksiin.

5 POHDINTA JA ARVIOINTI

5.1 Testausasetelma ja mittarit

Toteutetun RAG-chatbotin toimivuutta ja suorituskykyä arvioitiin tekemällä kymmenen testikyselyä tuotantoympäristössä. Testikysymykset valittiin kattamaan tyypilliset käyttötapaukset, kuten palvelun kuvaus, hinnoitteluun liittyvät kysymykset, ATS-käsitteiden selittäminen sekä ohjeistus palvelun käyttöön. Lisäksi testattiin asiakaspalveluun liittyviä kysymyksiä, englanninkielinen kysymys sekä tarkoituksella aiheeseen kuulumaton kysymys, jonka avulla arvioitiin järjestelmän kykyä rajata vastauksensa palvelun kontekstiin.

Jokaisesta testikyselystä mitattiin kokonaisvasteaika sekä sen jakautuminen kolmeen vaiheeseen:

- Kysymyksen vektorointi
- Relevanttien tekstikatkelmien haku tietokannasta
- Vastauksen generointi kielimallilla

Näiden mittausten avulla voitiin arvioida sekä järjestelmän reagointinopeutta että yksittäisten vaiheiden vaikutusta kokonaisvasteaikaan.

5.2 Vastauksen laatu ja rajaus

Testitulokset osoittivat, että chatbot tuottaa palveluun liittyviin kysymyksiin sisällöllisesti johdonmukaisia vastauksia, jotka perustuvat tietopankin sisältöön. Erityisesti selkeät ja yksiselitteiset kysymykset, kuten ATS-käsitteen määrittely tai palvelun toiminnan kuvaaminen, johtivat korkeampiin samankaltaisuusarvoihin ja täsmällisiin vastauksiin.

Aiheeseen kuulumaton kysymys (“Mikä on Suomen pääkaupunki?”) tuotti selvästi matalamman samankaltaisuusarvon kuin muut testit ja johti vastauksen rajaamiseen. Chatbot kieltäytyi vastaamasta kysymykseen ja ohjasi käyttäjän palvelun

kontekstiin. Tämä on keskeinen ominaisuus järjestelmän luotettavuuden kannalta, sillä se vähentää riskiä tuottaa virheellisiä tai spekulatiivisia vastauksia.

Englanninkielinen testikysely osoitti, että järjestelmä ei kaikissa tilanteissa vastaa käyttäjän kielellä, vaan muodosti vastauksen suomeksi. Tämä havainto ei vaikuttanut vastauksen sisällölliseen oikeellisuuteen, mutta tunnistaa kehityskohteen kieliohjauksessa, joka vaatii jatkossa promtien vahvistamisen, erillisen kielen tunnistamisen tai kielikohtaisen tietopankin.

5.3 Suorituskyky

Testien perusteella chatbotin keskimääräinen vasteaika oli noin 3,8 sekuntia. Nopein mitattu vastaus saatiin noin 1,4 sekunnissa ja hitain noin 7,9 sekunnissa. Vasteaikojen vaihtelu riippui pääasiassa vastauksen generointivaiheesta.

Kuvassa 18 on esitetty testikyselyiden vasteajat ja niiden jakautuminen eri vaiheisiin. Kuvasta nähdään, että tietokantahaku on kaikissa testitapauksissa nopeaa ja pysyy selvästi alle yhden sekunnin, kun taas kysymyksen vektorointi ja erityisesti vastauksen generointi muodostavat suurimman osan kokonaisvasteajasta. Vastauksen generointi on selvästi järjestelmän ajallisesti raskain vaihe.

Testi	Kysely	Upotus	Haku	Generointi	Yhteensä	Samankaltaisuus
1	Mikä on SkillResume?	1253	363	3477	5094	0.79
2	Paljonko palvelu maksaa?	1786	235	2761	4782	0.49
3	Mitä ATS tarkoittaa?	321	200	2120	2642	0.69
4	Miten luon CV:n?	266	98	2865	3229	0.60
5	Mikä on Suomen pääkaupunki?	1224	226	792	2243	0.21
6	Millaisia CV-pohjia?	305	116	7494	7916	0.50
7	What is SkillResume? (EN)	209	132	2214	2556	0.71
8	Miten saan apua?	912	148	2976	4036	0.40
9	Toimiiko mobiilissa?	207	120	1043	1371	0.36
10	Tekoälyn hyödyt CV:ssä?	1213	126	2889	4229	0.53

KUVA 18. RAG-chatbotin vasteajat testikyselyittäin (millisekuntia).

Vasteajan jakautumista prosessin eri vaiheisiin tarkastellaan tarkemmin kuvassa 19. Kuvan perusteella vektorihaku muodostaa vain pienen osan kokonaisvasteajasta, kun taas vastauksen generointi kattaa noin kolme neljäsosaa koko vasteajasta. Tämä osoittaa, että tietokantakerros ei ole suorituskyvyn pullonkaula, vaan suurin viive syntyy kielimallin tuottamasta vastauksesta.

Vaihe	Keskiarvo	Nopein	Hitain	Osuus kokonaisajasta
Upotus (embed_ms)	770 ms	207 ms	1786 ms	20%
Vektorihaku (retrieve_ms)	176 ms	98 ms	363 ms	5%
Generointi (generate_ms)	2863 ms	792 ms	7494 ms	75%
Yhteensä	**3810 ms**	**1371 ms**	**7916 ms**	100%

KUVA 19. Vasteajan jakautuminen prosessin vaiheisiin.

Vasteajat ovat pääosin hyväksyttävällä tasolla, mutta yksittäiset pidemmät viiveet voivat heikentää käyttäjäkokemusta. Tämän vuoksi vastauksen generointivaihe nousee järjestelmän keskeiseksi kehityskohteeksi. Generointivaiheen kokonaiskestoon vaikuttavat erityisesti käytettävän kielimallin aiheuttama latenssi sekä verkon viiveet.

5.4 Kustannukset ja skaalautuvuus

Yhden chatbot-viestin käsittely muodostuu kahdesta pääkustannuserästä: kysymyksen vektoroinnista ja vastauksen generoinnista kielimallilla. Koska konteksti rajataan muutamaan relevanttiin tekstikatkelmaan ja vastaukset pidetään suhteellisen lyhyinä, yksittäisen viestin kustannus jää hyvin pieneksi.

Kuvassa 20 on esitetty yhden viestin kustannusten jakautuminen eri vaiheiden välillä. Kuvasta nähdään, että kysymyksen vektorointi muodostaa kustannuksista vain marginaalisen osuuden, kun taas suurin osa kustannuksista syntyy kielimallin käyttämisestä vastauksen muodostamiseen.

Komponentti	Tokenit	Kustannus
Kyselyn upotus	~15	$0.0000003
LLM-syöte (prompt + konteksti)	~620	$0.000093
LLM-vastaus	~80	$0.000048
Yhteensä		**~$0.00014**

KUVA 20. Yhden viestin kustannusten jakautuminen.

Kustannustehokkuus mahdollistaa järjestelmän käytön myös suuremmilla käyttömäärillä ilman merkittäviä operatiivisia kustannuksia. Kustannusten maltillisuuteen vaikuttaa myös se, että testeissä ei hyödynnetty laajaa keskusteluhistoriaa vastauksen muodostamisessa, vaan vastaukset perustuivat yksittäiseen käyttäjäkysymykseen ja rajattuun tietopankkikontekstiin. Tämä rajoittaa syötteen pituutta ja vähentää kielimallin käyttökustannuksia.

Skaalautuvuuden näkökulmasta ratkaisevaa on se, että vektorihaku pysyy nopeana ja edullisena myös tietopankin kasvaessa, jolloin järjestelmän kokonaiskuormitus kohdistuu pääasiassa vastauksen generointiin.

5.5 Kehitysideat

Toteutuksen ja testitulosten perusteella tunnistettiin useita jatkokehityskohteita. Tietopankin ylläpito on nykyisessä ratkaisussa hallittua, mutta manuaalista. Jatkossa tietopankin päivittämistä voisi helpottaa erillinen ylläpitonäkymä, jonka avulla sisältöä voidaan muokata ja julkaista ilman erillisiä skriptejä.

Tällä hetkellä metatieto tuki on olemassa järjestelmässä, mutta sitä ei hyödynnetä toteutuksessa. Metatietojen hyödyntäminen tarjoaa selkeän mahdollisuuden parantaa vastausten jäljitettävyyttä. Metatietojen avulla käyttäjälle voitaisiin esimerkiksi näyttää, mihin tietopankin osioon vastaus perustuu, mikä lisäisi läpinäkyvyyttä ja luottamusta järjestelmään.

Prompt-ohjeistus vaikuttaa suoraan vastausten laatuun, tyyliin ja rajaukseen. Promptien versiointi ja vaihtoehtoisten ohjeistusten kokeilu mahdollistaisivat

hallitun kehittämisen sekä erilaisten vastaustapojen vertailun ilman muutoksia järjestelmän muuhun toimintaan.

OpenAI-rajapinnan päivittäminen uudempaan Responses API -rajapintaan mahdollistaisi streaming output -toiminnallisuuden hyödyntämisen järjestelmässä. Tämä tarkoittaisi, että vastauksen muodostuminen voitaisiin esittää käyttäjälle asteittain heti generoinnin alkaessa, mikä parantaisi koettua vasteaikaa ja käyttäjäkokemusta. Lisäksi rajapinnan päivittäminen tukisi järjestelmän pitkäjänteistä ylläpidettävyyttä ja yhteensopivuutta tulevien mallipäivitysten kanssa.

Tärkeää olisi myös kerätä järjestelmällisesti palautetta chatbotin tuottamista vastauksista. Kehitysideana järjestelmään voitaisiin toteuttaa käyttäjille mahdollisuus arvioida botin vastauksia esimerkiksi yksinkertaisella asteikolla tai palautepainikkeilla. Tällainen palaute mahdollistaisi vastausten laadun seuraamisen todellisessa käyttötilanteessa sekä tukisi tietopankin ja promptien jatkuvaa kehittämistä. Lisäksi palautedatan avulla voitaisiin tunnistaa kysymystyyppejä tai aihealueita, joissa vastaukset jäävät puutteellisiksi.

6 JOHTOPÄÄTÖKSET

Toteutettu RAG-pohjainen chatbot täytti sille asetetut tavoitteet esimerkkitoteutuksena. Järjestelmä kykenee hakemaan relevantin tiedon tietopankista, tuottamaan siihen perustuvia vastauksia ja rajaamaan aiheeseen kuulumattomat kysymykset. Testitulokset osoittavat, että tietokantahaku toimii tehokkaasti ja että suurin kehityspotentiaali liittyy vastauksen generointiin ja käyttäjäkokemuksen parantamiseen.

Työn aikana kävi ilmi, että RAG-pohjaisen chatbotin kehittäminen on ennen kaikkea kokonaisuuden hallintaa, ei yksittäisten teknisten ratkaisujen optimointia. Järjestelmän toimivuus syntyy useiden toisiinsa liittyvien suunnitteluratkaisujen yhteisvaikutuksesta, kuten siitä, miten tietopankin sisältö on rajattu, miten tieto on jäsennelty hyödynnettävään muotoon ja millä tavoin kielimallia ohjataan käyttämään saatavilla olevaa kontekstia. Tämä korostaa, että RAG-ratkaisun laatu rakentuu pitkälti suunnittelun ja ohjauksen kautta.

Kokonaisuutena opinnäytetyö syvensi ymmärrystä suurten kielimallien käytännön hyödyntämisestä osana tietojärjestelmää. Työ osoitti, että RAG-menetelmä tarjoaa joustavan ja kustannustehokkaan tavan yhdistää kielimallit organisaatiokohtaiseen tietoon, mutta samalla se vaatii huolellista suunnittelua, jatkuvaa ylläpitoa ja iteratiivista kehittämistä. Toteutettu esimerkkiratkaisu toimii hyvänä lähtökohtana jatkokehitykselle ja laajemmalle käyttöönotolle vastaavanlaisissa sovelluksissa.

LÄHTEET

Ali, U. 2024. The Power of Embeddings for Semantic Search. Medium. Luettavissa: https://medium.com/@umairali.khan/the-power-of-embeddings-for-semantic-search-8883f3fe8ba2. Luettu: 11.11.2025.

AWS s.a. What is RAG? Retrieval-Augmented Generation Options. AWS Prescriptive Guidance. Luettavissa: https://docs.aws.amazon.com/prescriptive-guidance/latest/retrieval-augmented-generation-options/what-is-rag.html. Luettu: 09.11.2025.

Bhat, S. R., Rudat, M., Spiekermann, J. & Flores-Herr, N. 2025. Rethinking Chunk Size For Long-Document Retrieval: A Multi-Dataset Analysis. arXiv:2505.21700. Luettavissa: https://arxiv.org/abs/2505.21700. Luettu: 22.11.2025.

Bommasani, R., ym. 2021. On the Opportunities and Risks of Foundation Models. Stanford Center for Research on Foundation Models. arXiv:2108.07258. Luettavissa: https://arxiv.org/abs/2108.07258. Luettu: 09.11.2025.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P. & Amodei, D. 2020. Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems (NeurIPS 2020). arXiv:2005.14165. Luettavissa: https://arxiv.org/abs/2005.14165. Luettu: 06.01.2026.

Bronsdon, C. 9.3.2025. Data processing steps for RAG precision & performance. Galileo AI Blog. Luettavissa: https://galileo.ai/blog/data-processing-steps-rag-precision-performance. Luettu: 09.11.2025.

Brucks, A. C., Fazio, R. H., Touroutoglou, A. & Beukema, S. 2025. Prompt architecture induces methodological artifacts in large language models. PLOS ONE, 20(1), e0319159. Luettavissa: https://doi.org/10.1371/journal.pone.0319159. Luettu: 23.11.2025.

Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., Tonellotto, N. & Silvestri, F. 2024. The Power of Noise: Redefining Retrieval for RAG Systems. SIGIR '24: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, 719–729. Association for Computing Machinery (ACM). Luettavissa: https://doi.org/10.1145/3626772.3657834. Luettu: 09.11.2025.

Gozzi, M. & Di Maio, G. 2024. Comparative Analysis of Prompt Strategies for Large Language Models: Single-Task vs. Multitask Prompts. Electronics, 13(23), 4712. Luettavissa: https://doi.org/10.3390/electronics13234712. Luettu: 23.11.2025.

Holdsworth, J. s.a. Unlocking the power of chatbots: Key benefits for businesses and customers. IBM Think. Luettavissa: https://www.ibm.com/think/insights/unlocking-the-power-of-chatbots-key-benefits-for-businesses-and-customers. Luettu: 04.11.2025.

Huyen, C. 2024. AI Engineering. O'Reilly Media. Luettavissa: https://www.oreilly.com/library/view/ai-engineering/9781098166298/. Luettu: 09.11.2025.

Kosisochukwu, M. 21.01.2025. What is Supabase: A Review of Serverless Database Features. Bejamas. Luettavissa: https://bejamas.com/hub/serverless-database/supabase. Luettu: 16.12.2025.

Laban, P., Lyu, Q., Bommasani, R., Yu, H. & Liang, P. 2024. Summary of a Haystack: A Challenge to Long-Context LLMs and RAG Systems. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024). Association for Computational Linguistics. Luettavissa: https://aclanthology.org/2024.emnlp-main.552/. Luettu: 09.11.2025.

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V. & Zettlemoyer, L. 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. arXiv:2005.11401. Luettavissa: https://arxiv.org/abs/2005.11401. Luettu: 09.11.2025.

Malkov, Y. A. & Yashunin, D. A. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836. Luettavissa: https://arxiv.org/abs/1603.09320. Luettu: 06.01.2026.

McKinsey & Company 30.10.2024. What is retrieval-augmented generation (RAG)? McKinsey Explainers. Luettavissa: https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-retrieval-augmented-generation-rag. Luettu: 09.11.2025.

Meredith, S. & Russi, E. 2025. What is vector search? IBM. Luettavissa: https://www.ibm.com/think/topics/vector-search. Luettu: 10.10.2025.

Merritt, R. 31.1.2025. What Is Retrieval-Augmented Generation, aka RAG?NVIDIA Blog. Luettavissa: https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/. Luettu: 15.12.2025.

MIT News 2023. Explained: Generative AI. Massachusetts Institute of Technology. Luettavissa: https://news.mit.edu/2023/explained-generative-ai-1109. Luettu: 19.11.2025.

Nebuly 2.8.2024. LLM system prompt vs user prompt. Luettavissa: https://www.nebuly.com/blog/llm-system-prompt-vs-user-prompt. Luettu: 08.11.2025.

NNLM s.a. Generative Artificial Intelligence. National Network of Libraries of Medicine. Luettavissa: https://www.nnlm.gov/guides/data-thesaurus/generative-artificial-intelligence. Luettu: 24.11.2025.

OpenAI s.a. Miksi kielimallit hallusinoivat? Luettavissa: https://openai.com/fi-FI/index/why-language-models-hallucinate/. Luettu: 08.11.2025.

PostgreSQL Global Development Group s.a. SQL Functions. PostgreSQL Documentation. Luettavissa: https://www.postgresql.org/docs/current/sql-createfunction.html. Luettu: 06.01.2026.

Supabase s.a. pgvector extension. Supabase Documentation. Luettavissa: https://supabase.com/docs/guides/database/extensions/pgvector. Luettu: 16.12.2025.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. 2017. Attention Is All You Need. Advances in Neural Information Processing Systems 30 (NeurIPS 2017). arXiv:1706.03762. Luettavissa: https://arxiv.org/abs/1706.03762. Luettu: 08.11.2025.